

LAB # 07

SQL Wildcards & Implementation of Databases Identifiers using Query Language

Lab Objective:

To understand SQL wild cards and primary and foreign key in SQL.

Software Tool:

MySql Workbench

SQL Wildcards

SQL wildcards can be used when searching for data in a database. SQL wildcards can substitute for one or more characters when searching for data in a database. SQL wildcards must be used with the SQL LIKE operator. With SQL, the following wildcards can be used:

Wildcard	Description
%	A substitute for zero or more characters
_	A substitute for exactly one character
[charlist]	Any single character in charlist
[^charlist] or [!charlist]	Any single character not in charlist

SQL Wildcard Examples

We have the following "Persons" table:

P_Id	LastName	FirstName	Addres	City
1	Hansen	Christ	Timoteivn 10	Sandness
2	Svendson	Tove	Borgvn 23	Sandness
3	Pettersen	Michael	Storgt 20	Stavanger
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jackob	Nissesetien	Sandness

Using the % Wildcard

Now we want to select the persons living in a city that starts with "sa" from the "Persons" table.

We use the following SELECT statement:

```
SELECT * FROM Persons
WHERE City LIKE 'sa%'
```

The result-set will look like this:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandness
2	Svendson	Tove	Borgvn 23	Sandness

Next, we want to select the persons living in a city that contains the pattern "nes" from the "Persons" table.

We use the following SELECT statement:

```
SELECT * FROM Persons
WHERE City LIKE '%nes%'
```

The result-set will look like this:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes

Using the _ Wildcard

Now we want to select the persons with a first name that starts with any character, followed by "ove" from the "Persons" table.

We use the following SELECT statement:

```
SELECT * FROM Persons
WHERE FirstName LIKE '_ove'
```

The result-set will look like this:

P_Id	LastName	FirstName	Addres	City
2	Svendson	Tove	Borgvn 23	Sandness

Using the [charlist] Wildcard

Now we want to select the persons with a last name that starts with "b" or "s" or "p" from the "Persons" table.

We use the following SELECT statement:

```
SELECT * FROM Persons
WHERE LastName LIKE '[bsp]%'
```

The result-set will look like this:

P_Id	LastName	FirstName	Address	City
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

Next, we want to select the persons with a last name that do not start with "b" or "s" or "p" from the "Persons" table.

We use the following SELECT statement:

```
SELECT * FROM Persons
WHERE LastName NOT LIKE '[bsp]%'
```

The result-set will look like this:

P_Id	LastName	FirstName	Addres	City
1	Hansen	Christ	Timoteivn 10	Sandness
4	Nilsen	Johan	Bakken 2	Stavanger
5	Tjessem	Jackob	Nissesetien	Sandness

SQL PRIMARY KEY Constraint

The PRIMARY KEY constraint uniquely identifies each record in a table. Primary keys must contain UNIQUE values and cannot contain NULL values. A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).

SQL PRIMARY KEY on CREATE TABLE

The following SQL creates a PRIMARY KEY on the "ID" column when the "Persons" table is created:

SQL example:

```
CREATE TABLE Persons (  
  ID int NOT NULL PRIMARY KEY,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Age int  
);
```

To allow naming of a PRIMARY KEY constraint, and for defining a PRIMARY KEY constraint on multiple columns, use the following SQL syntax:

SQL example:

```
CREATE TABLE Persons (  
  ID int NOT NULL,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Age int,  
  CONSTRAINT PK_Person PRIMARY KEY (ID, LastName)  
);
```

Note: In the example above there is only ONE PRIMARY KEY (PK_Person). However, the VALUE of the primary key is made up of TWO COLUMNS (ID + LastName).

SQL PRIMARY KEY on ALTER TABLE

To create a PRIMARY KEY constraint on the "ID" column when the table is already created, use the following SQL:

SQL example:

```
ALTER TABLE Persons  
ADD PRIMARY KEY (ID);
```

To allow naming of a PRIMARY KEY constraint, and for defining a PRIMARY KEY constraint on multiple columns, use the following SQL syntax:

SQL example:

```
ALTER TABLE Persons  
ADD CONSTRAINT PK_Person PRIMARY KEY  
(ID, LastName);
```

Note: If you use the ALTER TABLE statement to add a primary key, the primary key column(s) must already have been declared to not contain NULL values (when the table was first created).

DROP a PRIMARY KEY Constraint

To drop a PRIMARY KEY constraint, use the following SQL:

SQL example:

```
ALTER TABLE Persons  
DROP CONSTRAINT PK_Person;
```

SQL FOREIGN KEY Constraint

A FOREIGN KEY is a key used to link two tables together. A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table. The table containing the foreign key is called the child table, and the table containing the candidate key is called the referenced or parent table. Look at the following two tables:

"Persons" table:

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

"Orders" table:

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

Notice that the "PersonID" column in the "Orders" table points to the "PersonID" column in the "Persons" table. The "PersonID" column in the "Persons" table is the PRIMARY KEY in the "Persons" table. The "PersonID" column in the "Orders" table is a FOREIGN KEY in the "Orders" table. The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables. The FOREIGN KEY constraint also prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the table it points to.

Consider the structure of the following two tables.

CUSTOMERS Table

```
CREATE TABLE CUSTOMERS (  
    ID      INT                NOT NULL,  
    NAME    VARCHAR (20)       NOT NULL,  
    AGE     INT                NOT NULL,  
    ADDRESS CHAR (25) ,  
    SALARY  DECIMAL (18, 2) ,  
    PRIMARY KEY (ID)  
);
```

ORDERS Table

```
CREATE TABLE ORDERS (  
    ID INT NOT NULL,  
    CUSTOMER_ID INT,  
    DATE          DATETIME,  
    AMOUNT        double,  
    PRIMARY KEY (ID),  
    FOREIGN KEY (CUSTOMER_ID) references  
        CUSTOMERS (ID)  
);
```

If the ORDERS table has already been created and the foreign key has not yet been set, then use the syntax for specifying a foreign key by altering a table.

```
ALTER TABLE ORDERS  
ADD FOREIGN KEY (Customer_ID) REFERENCES  
CUSTOMERS (ID);
```

DROP a FOREIGN KEY Constraint

To drop a FOREIGN KEY constraint, use the following SQL syntax.

```
ALTER TABLE ORDERS  
DROP FOREIGN KEY;
```

For inserting values,

```
INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)  
VALUES ('5', 'ABC', '15', 'AA', '1000')  
INSERT INTO ORDERS (ID, DATE, CUSTOMER_ID, AMOUNT) VALUES  
('1', '6/12', '5', '1000')
```

LAB TASK:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Lulea	S-958 22	Sweden

1. Create the above table and do the following.

- Write an SQL statement that selects all Customers with a Country starting with the letter “s”.
- Write an SQL statement that selects all Customers with a Contact Name ending with the letter “s”.
- Write an SQL statement that selects all Customers with a City containing the pattern “ndo”.
- Write an SQL statement that selects all Customers with a City not containing the pattern “ndo”.
- Write an SQL statement that selects all customers with a **CustomerName** beginning with any letter followed by pattern “un”.
- Write an SQL statement that selects all Customers with a Customer Name starting with the letter “a” as well as letter “b”.

2. There are three tables **DeptInfo**, **ExamInfo** and **AdmissionInfo** of university database with the following schema:

DeptInfo (std_ID, std_Name, std_Address, std_dept)

ExamInfo (std_CNIC_No, std_ID, std_Name, std_GPA)

AdmissionInfo (enroll_No, std_Firstname, uni_id, enroll_date)

Write SQL query for the following questions:

- Create table with primary and foreign keys.
- Insert at least 5 rows in each table having CS and CE students.
- Display all departmental information of CE students.
- Display all student data whose GPA>3.0.

Conclusion:
